Міністерство освіти і науки України
Національний університет «Одеська політехніка»
Навчально-науковий інститут комп’ютерних систем
Кафедра інформаційних систем

Лабораторна робота № 4
з дисципліни: «Теорія алгоритмів»
Тема: «Робота з хеш-таблицями»

Варіант № 15

Виконав:
Студент групи АІ-254
Панькевич В. В.
Перевірили:
Смик С. Ю.
Арсірій О.О.

Одеса 2026

**Мета роботи:** набуття практичних навичок із проектування, реалізації, тестування та аналізу алгоритмів створення і обробки хеш-таблиць.

**Завдання:**

1. Спроектувати, реалізувати, протестувати та проаналізувати алгоритм створення відкритою хеш-таблиці при хешування з ланцюжками за варіантом завдання із Таблиці «Варіанти завдань», використовуючи хеш-функції побудовані за методом ділення h (K) = K mod 13 та множення h(k) = ⌊16 (kA mod 1)⌋

2. Виконати візуалізацію створення відкритої хеш-таблиці надати пояснення

3. Спроектувати, реалізувати, протестувати та проаналізувати алгоритм створення закритої хеш-таблиці при хешування з відкритою адресацією за варіантом завдання із Таблиці «Варіанти завдань», використовуючи хешфункції побудовані за методом ділення h (K) = K mod 13 та множення h(k)=⌊16 (kA mod 1)⌋

4. Виконати візуалізацію створення закритої хеш-таблиці надати пояснення

5. Показати на прикладі, що алгоритми хешування з ланцюжками та з відкритою адресацією мають середню ефективність сортування купою є нестійким та порівняти обчислювальну складність алгоритмів сортування купою та швидкого сортування за базовими операціями.

− знайдіть кількість порівнянь при пошуку (видаленні) будь-якого елемента із побудованих хеш-таблиць та вкажіть елемент з максимальною кількістю порівнянь у кожній хеш-таблиці

− підрахуйте середню кількість порівнянь для пошуку (видаленню) елемента в кожній хеш-таблиці

| Варіант | Послідовність |
|---|---|
| 15 | Хто грошима величається, той без душі зостачається. |

**Результати виконання завдання:**

**1. Підготовка вхідних даних та обчислення хеш-значень**

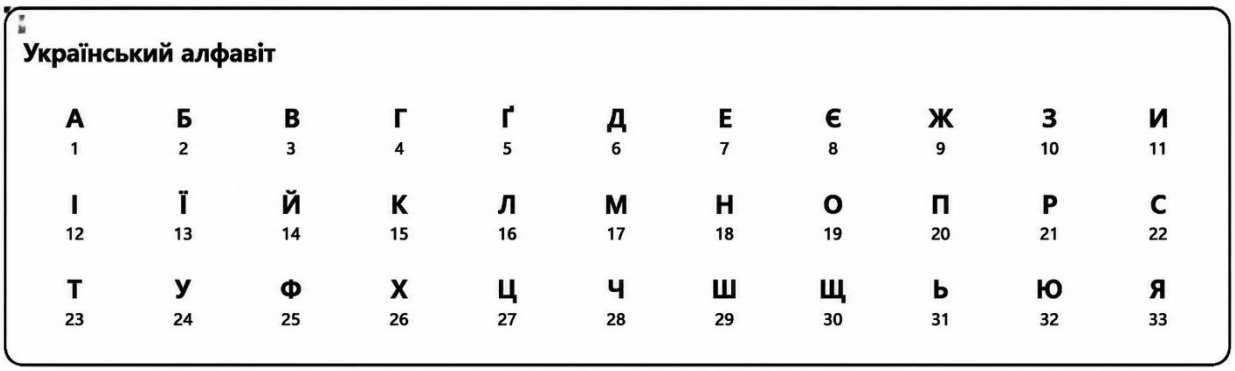
### 1.1. Визначення числових ключів

Для виконання лабораторної роботи використовується послідовність слів відповідно до варіанта №15: «Хто грошима величається, той без душі зостачається».

Для побудови хеш-таблиць необхідно перетворити кожне слово на числовий ключ. Для цього використовується український алфавіт, у якому кожній літері відповідає її порядковий номер від 1 до 33.

Числовий ключ K визначається як сума порядкових номерів усіх літер слова. Під час обчислення розділові знаки не враховуються, а всі літери переводяться у верхній регістр. Отримані числові ключі використовуються для обчислення хеш-значень за методами ділення та множення.

Для розрахунків використовуються такі порядкові номери літер:




**Таблиця 4.1 – Числові ключі слів варіанта №15**

| № | Слово | Сума номерів літер | Ключ K |
|---|---|---|---|
| 1 | ХТО | 26+23+19 | 68 |
| 2 | ГРОШИМА | 4+21+19+29+11+17+1 | 102 |
| 3 | ВЕЛИЧАЄТЬСЯ | 3+7+16+11+28+1+8+23+31+22+33 | 183 |
| 4 | ТОЙ | 23+19+14 | 56 |
| 5 | БЕЗ | 2+7+10 | 19 |
| 6 | ДУШІ | 6+24+29+12 | 71 |
| 7 | ЗОСТАЧАЄТЬСЯ | 10+19+22+23+1+28+1+8+23+31+22+33 | 221 |

### 1.2 Обчислення хеш-значень методами ділення та множення

Для визначення адрес розміщення елементів у хеш-таблицях використовуються два методи побудови хеш-функцій: метод ділення та метод множення.

За методом ділення хеш-значення визначається як залишок від ділення числового ключа K на розмір таблиці m = 13. За методом множення використовується розмір таблиці m = 16 та константа A ≈ 0,6180339887.

Для кожного слова варіанта №15 було обчислено хеш-значення за обома методами. Результати обчислень наведено в таблиці 4.2.

**Метод ділення**

$h_1(K) = K \bmod 13$

Наприклад, для слова «ХТО»: $h_1(68) = 68 \bmod 13 = 3$

**Метод множення**

$h_2(K) = \lfloor 16 * \{KA\} \rfloor$, де A = 0,6180339887, а {KA} – дробова частина добутку KA.

Для слова «ХТО»:

68 * 0,6180339887 = 42,0263112316

$h_2(68) = \lfloor 16 * 0,0263112316 \rfloor = \lfloor 0,4209797056 \rfloor = 0$

**Таблиця 4.2 – Результати обчислення хеш-значень**

| Слово | K | $h_1(K)$, mod 13 | $h_2(K)$, множення |
|---|---|---|---|
| ХТО | 68 | 3 | 0 |
| ГРОШИМА | 102 | 11 | 0 |
| ВЕЛИЧАЄТЬСЯ | 183 | 1 | 1 |
| ТОЙ | 56 | 4 | 9 |
| БЕЗ | 19 | 6 | 11 |
| ДУШІ | 71 | 6 | 14 |
| ЗОСТАЧАЄТЬСЯ | 221 | 0 | 9 |

За результатами таблиць 4.1 і 4.2 було визначено числові ключі семи слів варіанта №15 та обчислено їхні хеш-значення за методами ділення й множення.

Встановлено, що обидві хеш-функції призводять до виникнення колізій, коли різні слова отримують однакові індекси комірок. Для їх розв'язання надалі використовуватимуться два способи: хешування з роздільними ланцюжками та хешування з відкритою адресацією.

Отримані результати будуть використані для ручної побудови хеш-таблиць і перевірки правильності їх програмної реалізації мовою Python.

## 2. Відкрите хешування з роздільними ланцюжками

Відкрите хешування (Separate Chaining) є способом розв'язання колізій, за якого елементи з однаковими хеш-значеннями зберігаються в окремих ланцюжках, пов'язаних із відповідними комірками хеш-таблиці.

Для дослідження цього методу використовується послідовність із семи слів варіанта №15. Побудова хеш-таблиць здійснюється за допомогою двох хеш-функцій: методу ділення та методу множення. Слова додаються до ланцюжків у порядку їх появи у вихідній послідовності.

### 2.1 Побудова відкритої хеш-таблиці методом ділення

Для побудови відкритої хеш-таблиці методом ділення було використано числові ключі, визначені в таблиці 4.1, та відповідні хеш-значення з таблиці 4.2.

Кожне слово розміщується в комірці, індекс якої визначається залишком від ділення його числового ключа на 13. Якщо декілька слів мають однакове хеш-значення, вони додаються до одного ланцюжка. Результати ручної побудови хеш-таблиці наведено в таблиці 4.3.

**Таблиця 4.3 – Відкрита хеш-таблиця за методом ділення**

| Індекс | Ланцюжок слів |
|---|---|
| 0 | ЗОСТАЧАЄТЬСЯ |
| 1 | ВЕЛИЧАЄТЬСЯ |

| | |
|---|---|
| 2 | – |
| 3 | ХТО |
| 4 | ТОЙ |
| 5 | – |
| 6 | БЕЗ → ДУШІ |
| 7 | – |
| 8 | – |
| 9 | – |
| 10 | – |
| 11 | ГРОШИМА |
| 12 | – |

### 2.2 Побудова відкритої хеш-таблиці методом множення

Під час побудови відкритої хеш-таблиці методом множення для визначення адрес комірок використано хеш-функцію з розміром таблиці 16 та константою A ≈ 0,6180339887.

Обчислені хеш-значення наведено в таблиці 4.2. У разі збігу адрес декілька слів розміщуються в одному ланцюжку зі збереженням порядку їх додавання. Результати ручної побудови хеш-таблиці методом множення наведено в таблиці 4.4.

**Таблиця 4.4 – Відкрита хеш-таблиця за методом множення**

| Індекс | Ланцюжок слів |
|---|---|
| 0 | ХТО → ГРОШИМА |
| 1 | ВЕЛИЧАЄТЬСЯ |
| 2 | – |
| 3 | – |
| 4 | – |
| 5 | – |
| 6 | – |
| 7 | – |
| 8 | – |
| 9 | ТОЙ → ЗОСТАЧАЄТЬСЯ |

| | |
|---|---|
| 10 | – |
| 11 | БЕЗ |
| 12 | – |
| 13 | – |
| 14 | ДУШІ |
| 15 | – |

### 2.3 Графічна візуалізація відкритих хеш-таблиць

Для наочного представлення результатів ручної побудови відкритих хеш-таблиць було створено графічні схеми. На рисунку 4.1 показано розміщення слів за методом ділення, а на рисунку 4.2 – за методом множення.

Під час використання методу ділення утворився один ланцюжок із двох слів, тоді як за методом множення – два таких ланцюжки. В обох випадках колізії розв'язуються шляхом збереження елементів з однаковими хеш-значеннями у відповідних списках.

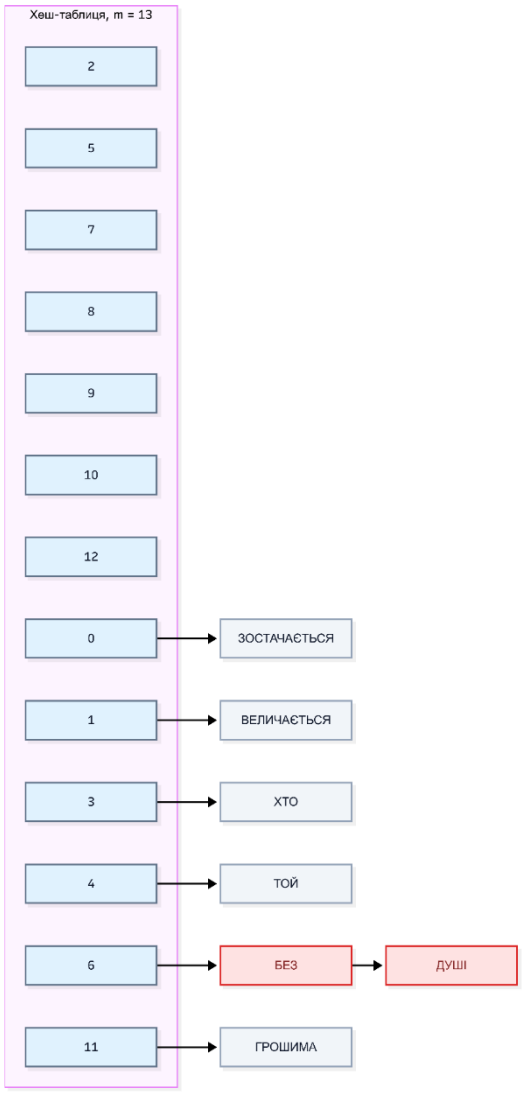


Рисунок 4.1 – Відкрита хеш-таблиця за методом ділення

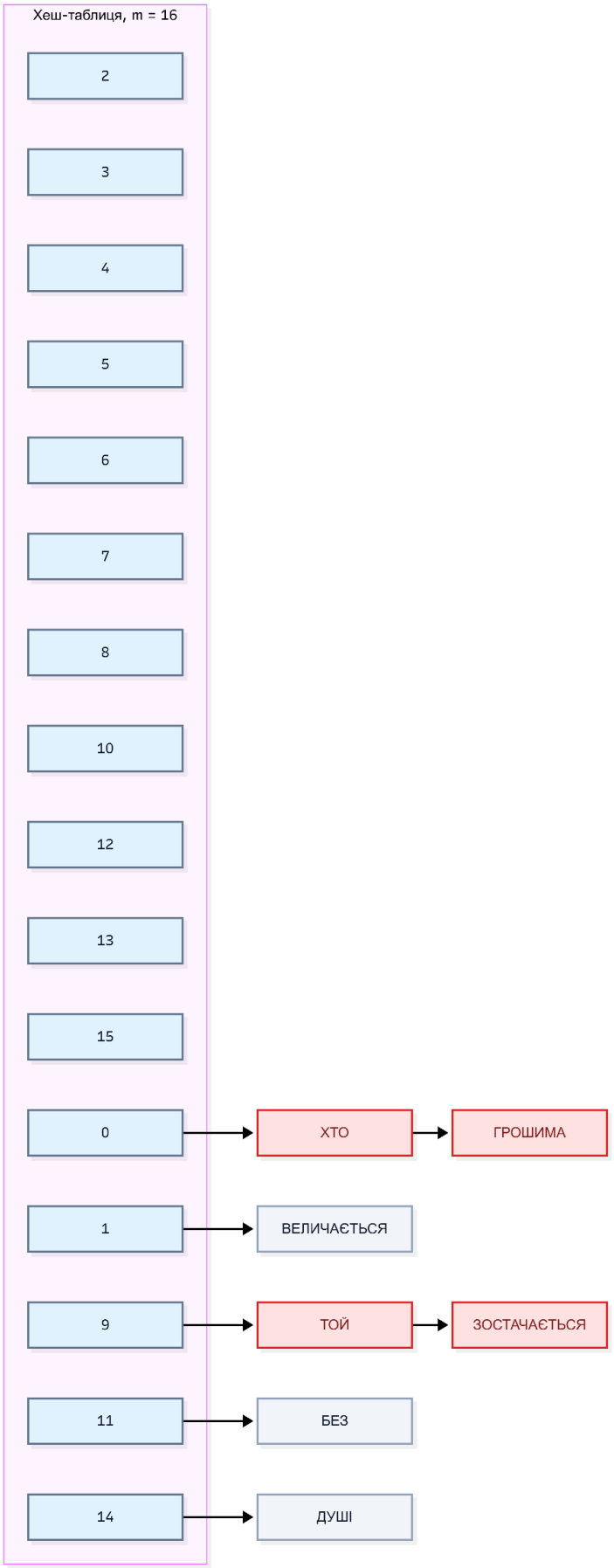



Рисунок 4.2 – Відкрита хеш-таблиця за методом множення

### 2.4 Реалізація відкритого хешування мовою Python

Для перевірки результатів ручної побудови відкритих хеш-таблиць було розроблено програмну реалізацію методу хешування з роздільними ланцюжками мовою Python.

Програма використовує послідовність слів варіанта №15 та два способи обчислення хеш-значень: метод ділення і метод множення. Числові ключі слів визначаються як сума порядкових номерів літер українського алфавіту. У разі виникнення колізій елементи додаються до відповідних ланцюжків зі збереженням порядку їх появи.

Python-код реалізації відкритого хешування з виведенням отриманих таблиць наведено в Лістингу 4.1.

**Лістинг 4.1 – Python-код реалізації відкритого хешування з роздільними ланцюжками**

```
import math

# Розміри хеш-таблиць
M_DIV = 13
M_MUL = 16

# Константа для методу множення
A = 0.6180339887

# Послідовність слів варіанта №15
WORDS = [
    "ХТО", "ГРОШИМА", "ВЕЛИЧАЄТЬСЯ",
    "ТОЙ", "БЕЗ", "ДУШІ", "ЗОСТАЧАЄТЬСЯ"
]

# Український алфавіт
ALPHABET = "АБВГҐДЕЄЖЗИІЇЙКЛМНОПРСТУФХЦЧШЩЬЮЯ"

# Словник порядкових номерів літер
LETTER_POSITIONS = {
    char: i + 1 for i, char in enumerate(ALPHABET)
}


def calculate_key(word):
    """
    Обчислення числового ключа слова.
    K = сума порядкових номерів літер.
    """
```

```
    key = 0

    # Обробка кожної літери слова
    for char in word.upper():
        key += LETTER_POSITIONS.get(char, 0)

    return key


def division_hash(key):
    """Метод ділення: h(K) = K mod 13."""
    return key % M_DIV


def multiplication_hash(key):
    """
    Метод множення:
    h(K) = floor(16 * (KA mod 1)).
    """
    fractional_part = (key * A) % 1
    return math.floor(M_MUL * fractional_part)


def build_open_hash_table(words, m, hash_function):
    """
    Побудова відкритої хеш-таблиці
    методом роздільних ланцюжків.
    """
    # Ініціалізація порожніх ланцюжків
    hash_table = [[] for _ in range(m)]

    # Обробка кожного слова
    for word in words:

        # Обчислення числового ключа
        key = calculate_key(word)

        # Визначення адреси комірки
        address = hash_function(key)

        # Додавання слова до відповідного ланцюжка
        hash_table[address].append(word)

    return hash_table


def display_hash_table(table, method):
    """Виведення хеш-таблиці."""
    print(f"\n--- {method} ---")

    for i, chain in enumerate(table):
        print(f"Індекс {i:02d}: {chain}")


# Побудова таблиці методом ділення
table_div = build_open_hash_table(
    WORDS, M_DIV, division_hash
)

display_hash_table(
    table_div, "Метод ділення (M=13)"
)

# Побудова таблиці методом множення
```

```
table_mul = build_open_hash_table(
    WORDS, M_MUL, multiplication_hash
)

display_hash_table(
    table_mul, "Метод множення (M=16)"
)
```

---

У результаті виконання програмного коду, наведеного в Лістингу 4.1, було побудовано відкриті хеш-таблиці з роздільними ланцюжками за методами ділення та множення. Програма визначила хеш-адреси слів варіанта №15 і розмістила їх у відповідних комірках таблиць.

Результати виконання відкритого хешування за методом ділення наведено в таблиці 4.5, а за методом множення – у таблиці 4.6.

**Таблиця 4.5 – Результати виконання відкритого хешування за методом ділення**

```
--- Метод ділення (M=13) ---
Індекс 00: ['ЗОСТАЧАЄТЬСЯ']
Індекс 01: ['ВЕЛИЧАЄТЬСЯ']
Індекс 02: []
Індекс 03: ['ХТО']
Індекс 04: ['ТОЙ']
Індекс 05: []
Індекс 06: ['БЕЗ', 'ДУШІ']
Індекс 07: []
Індекс 08: []
Індекс 09: []
Індекс 10: []
Індекс 11: ['ГРОШИМА']
Індекс 12: []
```

**Таблиця 4.6 – Результати виконання відкритого хешування за методом множення**

```
--- Метод множення (M=16) ---
Індекс 00: ['ХТО', 'ГРОШИМА']
Індекс 01: ['ВЕЛИЧАЄТЬСЯ']
Індекс 02: []
Індекс 03: []
Індекс 04: []
Індекс 05: []
Індекс 06: []
Індекс 07: []
Індекс 08: []
Індекс 09: ['ТОЙ', 'ЗОСТАЧАЄТЬСЯ']
Індекс 10: []
Індекс 11: ['БЕЗ']
Індекс 12: []
```

```
Індекс 13: []
Індекс 14: ['ДУШІ']
Індекс 15: []
```

**Аналіз результатів:**

Порівняння результатів ручної побудови відкритих хеш-таблиць (таблиці 4.3 і 4.4) та програмного виконання (таблиці 4.5 і 4.6) показало їх повний збіг. В обох випадках усі сім слів варіанта №15 було правильно розміщено у відповідних комірках.

За методом ділення виникла одна колізія між словами «БЕЗ» і «ДУШІ», які потрапили до комірки 6. За методом множення виникли дві колізії: між словами «ХТО» і «ГРОШИМА» в комірці 0 та між словами «ТОЙ» і «ЗОСТАЧАЄТЬСЯ» в комірці 9.

Отримані результати підтверджують правильність реалізації відкритого хешування з роздільними ланцюжками. Усі колізії було розв'язано шляхом збереження слів з однаковими хеш-значеннями у відповідних списках.

### 3. Побудова закритих хеш-таблиць із відкритою адресацією

Закрите хешування (Open Addressing) є способом розв'язання колізій, за якого всі елементи зберігаються безпосередньо в комірках хеш-таблиці. На відміну від відкритого хешування з роздільними ланцюжками, кожна комірка може містити лише один елемент.

Для вирішення колізій використовується метод лінійного пробування (Linear Probing). Якщо комірка, визначена хеш-функцією, вже зайнята, послідовно перевіряються наступні комірки до знаходження вільної. У разі досягнення кінця таблиці перевірка продовжується з її початку.

Для дослідження закритого хешування використовується послідовність слів варіанта №15 та дві хеш-функції, результати обчислення яких наведено в таблиці 4.2. Побудова хеш-таблиць здійснюється методами ділення та множення.

Для лінійного пробування застосовуємо формулу: $h(K, i)=(h(K) + i) \bmod m$, де $h(K)$ – початкова адреса, $i = 0, 1, 2, \ldots$) – номер спроби, $m$ – розмір таблиці.

### 3.1 Побудова закритої хеш-таблиці методом ділення

Для побудови закритої хеш-таблиці методом ділення використано числові ключі та хеш-значення, отримані в таблицях 4.1 і 4.2. Розмір хеш-таблиці становить 13 комірок.

Кожне слово вставляється за адресою, визначеною хеш-функцією. Якщо відповідна комірка вже зайнята, виконується лінійне пробування з переходом до наступної комірки. Послідовність вставлення елементів та результати ручної побудови хеш-таблиці наведено в таблиці 4.7.

**Таблиця 4.7 – Хід побудови закритої хеш-таблиці за методом ділення**

| № | Слово | Початкова адреса | Перевірені комірки | Кінцева адреса |
|---|---|---|---|---|
| 1 | ХТО | 3 | 3 | 3 |
| 2 | ГРОШИМА | 11 | 11 | 11 |
| 3 | ВЕЛИЧАЄТЬСЯ | 1 | 1 | 1 |
| 4 | ТОЙ | 4 | 4 | 4 |
| 5 | БЕЗ | 6 | 6 | 6 |
| 6 | ДУШІ | 6 | 6 → 7 | 7 |
| 7 | ЗОСТАЧАЄТЬСЯ | 0 | 0 | 0 |

**Пояснення колізії**

Перші п'ять слів розміщуються без колізій.

Під час вставлення слова «ДУШІ» отримуємо: $h_1(71) = 71 \bmod 13 = 6$

Але комірка 6 уже містить слово «БЕЗ». Тому перевіряємо наступну комірку: $h_1(71,1) = (6+1) \bmod 13 = 7$

Комірка 7 вільна, тому слово «ДУШІ» розміщується саме там.

**Таблиця 4.8 – Закрита хеш-таблиця за методом ділення**

| Індекс | Слово |
|---|---|
| 0 | ЗОСТАЧАЄТЬСЯ |
| 1 | ВЕЛИЧАЄТЬСЯ |
| 2 | – |
| 3 | ХТО |
| 4 | ТОЙ |
| 5 | – |
| 6 | БЕЗ |
| 7 | ДУШІ |
| 8 | – |
| 9 | – |
| 10 | – |
| 11 | ГРОШИМА |
| 12 | – |

Таким чином, для методу ділення виникла одна колізія, розв'язана за допомогою одного додаткового кроку лінійного пробування.

### 3.2 Побудова закритої хеш-таблиці методом множення

Для побудови закритої хеш-таблиці методом множення використано хеш-функцію з розміром таблиці 16 та константою A ≈ 0,6180339887.

У разі виникнення колізій застосовується лінійне пробування, яке передбачає послідовний пошук наступної вільної комірки. При цьому адреса нового елемента може відрізнятися від початкового хеш-значення.

Результати ручного вставлення слів варіанта №15 до закритої хеш-таблиці методом множення наведено в таблиці 4.9.

**Таблиця 4.9 – Хід побудови закритої хеш-таблиці за методом множення**

| № | Слово | Початкова адреса | Перевірені комірки | Кінцева адреса |
|---|---|---|---|---|
| 1 | ХТО | 0 | 0 | 0 |

| | | | | |
|---|---|---|---|---|
| 2 | ГРОШИМА | 0 | 0 → 1 | 1 |
| 3 | ВЕЛИЧАЄТЬСЯ | 1 | 1 → 2 | 2 |
| 4 | ТОЙ | 9 | 9 | 9 |
| 5 | БЕЗ | 11 | 11 | 11 |
| 6 | ДУШІ | 14 | 14 | 14 |
| 7 | ЗОСТАЧАЄТЬСЯ | 9 | 9 → 10 | 10 |

**Пояснення колізій**

**Перша колізія – слово «ГРОШИМА»**

Початкова адреса дорівнює 0, але там уже міститься «ХТО». Перевіряємо наступну комірку: $h_2(102,1) = (0 + 1) \bmod 16 = 1$

Слово «ГРОШИМА» розміщується за індексом 1.

**Друга колізія – слово «ВЕЛИЧАЄТЬСЯ»**

Початкова адреса 1, однак ця комірка вже зайнята словом «ГРОШИМА», яке раніше було переміщене внаслідок колізії: $h_2(183,1) = (1 + 1) \bmod 16 = 2$

Слово «ВЕЛИЧАЄТЬСЯ» розміщується за індексом 2.

**Третя колізія – слово «ЗОСТАЧАЄТЬСЯ»**

Початкова адреса 9 зайнята словом «ТОЙ»:

$h_2(221,1) = (9 + 1) \bmod 16 = 10$

Отже, слово «ЗОСТАЧАЄТЬСЯ» розміщується в комірці 10.

**Таблиця 4.10 – Закрита хеш-таблиця за методом множення**

| Індекс | Слова |
|---|---|
| 0 | ХТО |
| 1 | ГРОШИМА |
| 2 | ВЕЛИЧАЄТЬСЯ |
| 3 | – |
| 4 | – |
| 5 | – |
| 6 | – |
| 7 | – |

| | |
|---|---|
| 8 | – |
| 9 | ТОЙ |
| 10 | ЗОСТАЧАЄТЬСЯ |
| 11 | БЕЗ |
| 12 | – |
| 13 | – |
| 14 | ДУШІ |
| 15 | – |

### 3.3 Аналіз результатів ручної побудови

За результатами ручної побудови закритих хеш-таблиць (таблиці 4.7–4.10) встановлено, що обидві хеш-функції забезпечують розміщення всіх семи слів варіанта №15 без втрати елементів.

Під час використання методу ділення виникла одна колізія, унаслідок якої слово «ДУШІ» було переміщено з комірки 6 до комірки 7. За методом множення виникли три колізії, які призвели до переміщення слів «ГРОШИМА», «ВЕЛИЧАЄТЬСЯ» та «ЗОСТАЧАЄТЬСЯ» до наступних вільних комірок.

Таким чином, застосування лінійного пробування дозволило успішно розв'язати всі колізії. Для заданої послідовності слів метод ділення потребував меншої кількості додаткових перевірок комірок, ніж метод множення.

### 3.4 Графічна візуалізація закритих хеш-таблиць

Для наочного представлення процесу побудови закритих хеш-таблиць було створено графічні схеми, які відображають розміщення елементів у комірках та застосування лінійного пробування під час виникнення колізій.

На рисунку 4.3 показано результати хешування за методом ділення, а на рисунку 4.4 – за методом множення. Стрілками позначено переходи між комірками під час пошуку вільного місця для вставлення елементів.




Рисунок 4.3 – Закрита хеш-таблиця за методом ділення

**Пояснення:** слово «ДУШІ» має початкову адресу 6, але вона зайнята словом «БЕЗ». Тому воно переміщується до вільної комірки 7.




Рисунок 4.4 – Закрита хеш-таблиця за методом множення

За результатами графічної візуалізації (рисунки 4.3 і 4.4) встановлено, що використання лінійного пробування дозволяє розмістити всі елементи без утворення ланцюжків. Кожне слово займає окрему комірку хеш-таблиці.

Під час хешування методом ділення виникла одна колізія, унаслідок якої слово «ДУШІ» було розміщено в наступній вільній комірці. За методом множення виникли три колізії, що призвели до зміни адрес слів «ГРОШИМА», «ВЕЛИЧАЄТЬСЯ» та «ЗОСТАЧАЄТЬСЯ».

Отримані схеми відповідають результатам ручної побудови закритих хеш-таблиць (таблиці 4.7-4.10) та демонструють особливості розв'язання колізій методом відкритої адресації.

### 3.5 Реалізація закритого хешування мовою Python

Для перевірки результатів ручної побудови закритих хеш-таблиць було розроблено програмну реалізацію хешування з відкритою адресацією мовою Python. Програма використовує послідовність слів варіанта №15 та два методи обчислення хеш-значень: ділення і множення.

Для розв'язання колізій застосовується лінійне пробування, за якого послідовно перевіряються комірки таблиці до знаходження вільного місця. У програмі реалізовано функції побудови та виведення хеш-таблиць, що дозволяє порівняти результати програмного виконання з результатами ручної побудови.

Python-код реалізації закритого хешування з відкритою адресацією наведено в Лістингу 4.2.

**Лістинг 4.2 – Python-код реалізації закритого хешування з відкритою адресацією**

```
import math

# Розміри хеш-таблиць
M_DIV = 13
M_MUL = 16

# Константа методу множення
```

```
A = 0.6180339887

# Послідовність слів варіанта №15
WORDS = [
    "ХТО", "ГРОШИМА", "ВЕЛИЧАЄТЬСЯ",
    "ТОЙ", "БЕЗ", "ДУШІ", "ЗОСТАЧАЄТЬСЯ"
]

# Український алфавіт
ALPHABET = "АБВГҐДЕЄЖЗИІЇЙКЛМНОПРСТУФХЦЧШЩЬЮЯ"

# Словник порядкових номерів літер
LETTER_POSITIONS = {
    char: i + 1 for i, char in enumerate(ALPHABET)
}


def calculate_key(word):
    """
    Обчислення числового ключа слова.
    K = сума порядкових номерів літер.
    """
    key = 0

    for char in word.upper():
        key += LETTER_POSITIONS.get(char, 0)

    return key


def division_hash(key):
    """Метод ділення: h(K) = K mod 13."""
    return key % M_DIV


def multiplication_hash(key):
    """
    Метод множення:
    h(K) = floor(16 * (KA mod 1)).
    """
    fractional_part = (key * A) % 1
    return math.floor(M_MUL * fractional_part)


def build_closed_hash_table(words, m, hash_function):
    """
    Побудова закритої хеш-таблиці
    з використанням лінійного пробування.
    """
    # Ініціалізація порожніх комірок
    hash_table = [None] * m

    # Обробка кожного слова
    for word in words:

        # Обчислення числового ключа
        key = calculate_key(word)

        # Визначення початкової адреси
        start_address = hash_function(key)

        # Лінійне пробування
        for i in range(m):
```

```
            # Обчислення поточної адреси
            address = (start_address + i) % m

            # Перевірка вільної комірки
            if hash_table[address] is None:

                # Вставлення слова
                hash_table[address] = word
                break

        else:
            # Якщо всі комірки зайняті
            print(f"Помилка: таблиця заповнена. "
                  f"Не вдалося додати: {word}")

    return hash_table


def display_hash_table(table, method):
    """Виведення закритої хеш-таблиці."""
    print(f"\n--- {method} ---")
    print("Індекс | Слово")
    print("-------|----------------")

    for i, item in enumerate(table):
        value = item if item is not None else "(NULL)"
        print(f"{i:02d}     | {value}")


# Побудова таблиці методом ділення
table_div = build_closed_hash_table(
    WORDS, M_DIV, division_hash
)

display_hash_table(
    table_div, "Метод ділення (M=13)"
)

# Побудова таблиці методом множення
table_mul = build_closed_hash_table(
    WORDS, M_MUL, multiplication_hash
)

display_hash_table(
    table_mul, "Метод множення (M=16)"
)
```

У результаті виконання програмного коду, наведеного в Лістингу 4.2, було побудовано закриті хеш-таблиці з відкритою адресацією за методами ділення та множення. Для розв'язання колізій застосовано лінійне пробування, що забезпечило розміщення всіх семи слів варіанта №15 у відповідних комірках.

Результати виконання закритого хешування за методом ділення наведено в таблиці 4.11, а за методом множення – у таблиці 4.12.

**Таблиця 4.11 – Результати виконання закритого хешування за методом ділення**

```
--- Метод ділення (M=13) ---
Індекс | Слово
-------|----------------
00     | ЗОСТАЧАЄТЬСЯ
01     | ВЕЛИЧАЄТЬСЯ
02     | (NULL)
03     | ХТО
04     | ТОЙ
05     | (NULL)
06     | БЕЗ
07     | ДУШІ
08     | (NULL)
09     | (NULL)
10     | (NULL)
11     | ГРОШИМА
12     | (NULL)
```

**Таблиця 4.12 – Результати виконання закритого хешування за методом множення**

```
--- Метод множення (M=16) ---
Індекс | Слово
-------|----------------
00     | ХТО
01     | ГРОШИМА
02     | ВЕЛИЧАЄТЬСЯ
03     | (NULL)
04     | (NULL)
05     | (NULL)
06     | (NULL)
07     | (NULL)
08     | (NULL)
09     | ТОЙ
10     | ЗОСТАЧАЄТЬСЯ
11     | БЕЗ
12     | (NULL)
13     | (NULL)
14     | ДУШІ
15     | (NULL)
```

**Аналіз результатів:**

Порівняння результатів ручної побудови закритих хеш-таблиць (таблиці 4.8 і 4.10) та програмного виконання (таблиці 4.11 і 4.12) показало їх повний збіг. В обох випадках усі сім слів варіанта №15 було розміщено без втрати елементів.

За методом ділення виникла одна колізія, унаслідок якої слово «ДУШІ» було переміщено з комірки 6 до комірки 7. За методом множення виникли три колізії,

що призвели до переміщення слів «ГРОШИМА», «ВЕЛИЧАЄТЬСЯ» та «ЗОСТАЧАЄТЬСЯ» до наступних вільних комірок.

Отримані результати підтверджують правильність реалізації закритого хешування з відкритою адресацією. Для заданої послідовності слів метод ділення забезпечив меншу кількість додаткових перевірок комірок під час вставлення порівняно з методом множення.

## 4. Порівняльний аналіз ефективності хешування

Для оцінювання ефективності розроблених алгоритмів було проведено порівняльний аналіз пошуку елементів у хеш-таблицях, побудованих методами ділення та множення з використанням роздільних ланцюжків і відкритої адресації.

Для кожної з чотирьох хеш-таблиць визначено кількість порівнянь, необхідних для успішного пошуку кожного слова варіанта №15. Одним порівнянням вважається перевірка шуканого слова на рівність слову, що зберігається в поточній комірці або елементі ланцюжка. Обчислення хеш-функції до кількості порівнянь не включається.

Отримані результати дозволяють визначити максимальну та середню кількість порівнянь і порівняти ефективність використаних методів хешування.

### 4.1 Визначення кількості порівнянь

При пошуку з роздільними ланцюжками перевіряємо елементи відповідного списку до знаходження потрібного слова.

При відкритій адресації перевіряємо комірки починаючи з початкової хеш-адреси, переходячи до наступних за необхідності.

Наприклад, для слова «ДУШІ» за методом ділення:

- Роздільні ланцюжки: комірка 6 містить БЕЗ → ДУШІ, тому потрібно 2 порівняння.

- Відкрита адресація: комірка 6 містить БЕЗ, а комірка 7 – ДУШІ, тому також потрібно 2 порівняння.

**Таблиця 4.13 – Порівняння кількості операцій пошуку в хеш-таблицях**

| Слово | Ланцюжки, ділення | Ланцюжки, множення | Адресація, ділення | Адресація, множення |
|---|---|---|---|---|
| ХТО | 1 | 1 | 1 | 1 |
| ГРОШИМА | 1 | 2 | 1 | 2 |
| ВЕЛИЧАЄТЬСЯ | 1 | 1 | 1 | 2 |
| ТОЙ | 1 | 1 | 1 | 1 |
| БЕЗ | 1 | 1 | 1 | 1 |
| ДУШІ | 2 | 1 | 2 | 1 |
| ЗОСТАЧАЄТЬСЯ | 1 | 2 | 1 | 2 |
| Усього | 8 | 9 | 8 | 10 |

### 4.2 Обчислення середньої кількості порівнянь

Середню кількість порівнянь для успішного пошуку визначаємо за формулою: $C_{\text{сер}} = \frac{\Sigma_{i=1}^{n} C_i}{n}$, де ($C_i$) – кількість порівнянь для пошуку $i$-го слова, $n$ = 7 – кількість слів.

1. Роздільні ланцюжки, метод ділення: $C_{\text{сер1}} = \frac{8}{7} \approx 1{,}14$
2. Роздільні ланцюжки, метод множення: $C_{\text{сер2}} = \frac{9}{7} \approx 1{,}29$
3. Відкрита адресація, метод ділення: $C_{\text{сер3}} = \frac{8}{7} \approx 1{,}14$
4. Відкрита адресація, метод множення: $C_{\text{сер4}} = \frac{10}{7} \approx 1{,}43$

### 4.3 Порівняння отриманих результатів

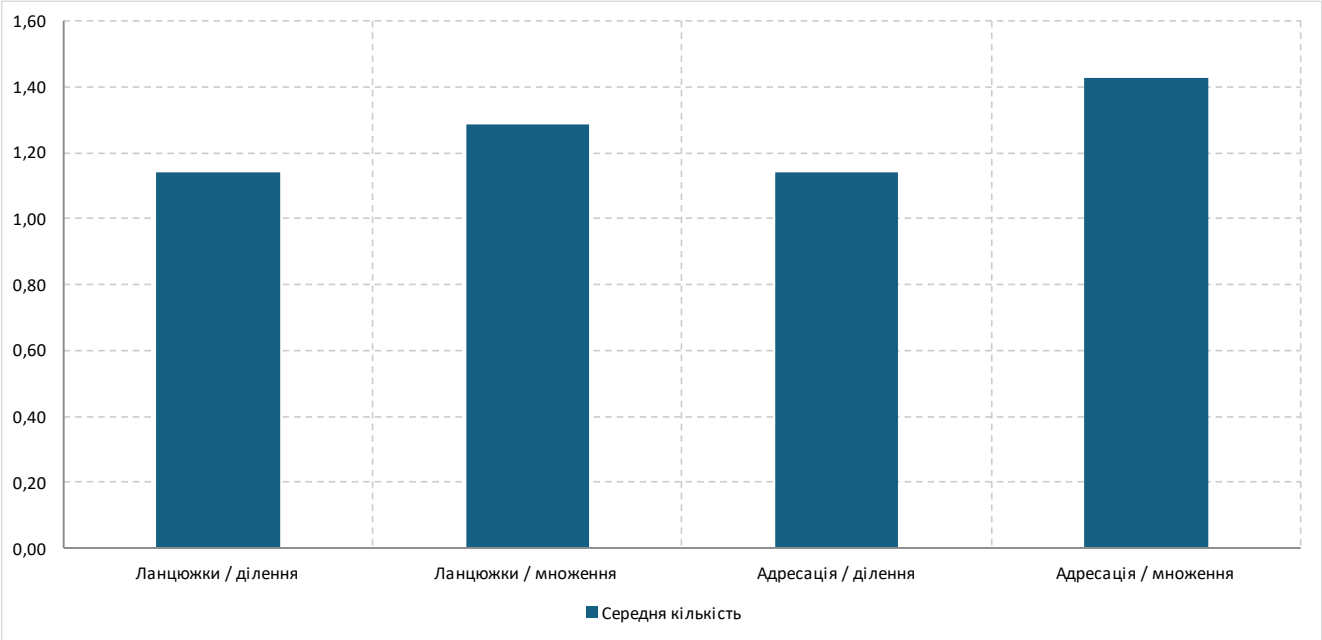



Рисунок 4.5 – Порівняння середньої кількості порівнянь при успішному пошуку елементів у хеш-таблицях.

**Таблиця 4.14 – Порівняння ефективності методів хешування**

| Метод хешування | Максимум порівнянь | Середня кількість |
|---|---|---|
| Ланцюжки, ділення | 2 | 1,14 |
| Ланцюжки, множення | 2 | 1,29 |
| Адресація, ділення | 2 | 1,14 |
| Адресація, множення | 2 | 1,43 |

**Аналіз результатів:**

За результатами порівняння (таблиці 4.13 і 4.14) встановлено, що для заданої послідовності слів варіанта №15 найменшу середню кількість порівнянь забезпечили хешування з роздільними ланцюжками та хешування з відкритою адресацією за методом ділення. В обох випадках цей показник становить 1,14 порівняння.

При використанні методу множення середня кількість порівнянь становила 1,29 для роздільних ланцюжків та 1,43 для відкритої адресації. Це пояснюється більшою кількістю колізій і необхідністю додаткових перевірок елементів під час пошуку.

Максимальна кількість порівнянь у всіх чотирьох хеш-таблицях становила 2. Для заданого набору даних метод ділення виявився ефективнішим за метод множення, оскільки забезпечив меншу середню кількість порівнянь.

Максимальна кількість порівнянь для кожного з чотирьох способів хешування становить 2. При використанні роздільних ланцюжків за методом ділення такої кількості порівнянь потребує пошук слова «ДУШІ», а за методом множення – слів «ГРОШИМА» та «ЗОСТАЧАЄТЬСЯ». Для відкритої адресації за методом ділення максимальна кількість порівнянь також досягається під час пошуку слова «ДУШІ», а за методом множення – слів «ГРОШИМА», «ВЕЛИЧАЄТЬСЯ» та «ЗОСТАЧАЄТЬСЯ».

У середньому пошук у хеш-таблицях за умови рівномірного розподілу ключів може виконуватися за час O(1). Проте в найгіршому випадку за наявності значної кількості колізій складність пошуку може зростати до O(n). Отримані результати демонструють вплив вибору хеш-функції та способу розв'язання колізій на ефективність пошуку елементів.

**Висновок:**

Під час виконання лабораторної роботи було досліджено, реалізовано та протестовано алгоритми створення й обробки хеш-таблиць із використанням роздільних ланцюжків (Separate Chaining) та відкритої адресації (Open Addressing). Для послідовності із семи слів варіанта №15 було визначено числові ключі та обчислено хеш-значення за методами ділення і множення.

Шляхом ручної побудови та графічної візуалізації досліджено особливості розміщення елементів у чотирьох хеш-таблицях і способи розв'язання колізій. Також було розроблено та протестовано програмні реалізації алгоритмів мовою Python. Результати програмного виконання повністю збіглися з результатами ручної побудови.

Проведений порівняльний аналіз показав, що для заданої послідовності слів найменшу середню кількість порівнянь під час успішного пошуку забезпечив

метод ділення – 1,14 для обох способів хешування. За методом множення цей показник становив 1,29 для роздільних ланцюжків та 1,43 для відкритої адресації. Максимальна кількість порівнянь у всіх досліджених таблицях дорівнювала 2.

Таким чином, було набуто практичних навичок побудови хеш-функцій, створення й візуалізації хеш-таблиць, розв'язання колізій та аналізу ефективності пошуку елементів. Отримані результати підтвердили, що ефективність хешування залежить від вибору хеш-функції, способу обробки колізій та особливостей вхідних даних.

**Посилання на GitHub:** https://github.com/Pankevich-Vyacheslav/Lab_4/tree/main

**Посилання на онлайн компілятор:** https://coddy.tech/playground/ru/python